

LAPORAN PRATIKUM

STRUKTUR DATA

“QUEUE“

PEKAN KE-4

disusun Oleh:

Aisyah Najmiatul Fauziah

NIM 2511533030

Dosen Pengampu: DR. WAHYUDI, S.T, M.T

Asisten Pratikum: Sherly Sukmadira



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya penulis dapat menyelesaikan laporan praktikum Struktur Data dengan judul **“Queue (Antrian)”** ini dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas praktikum mata kuliah Struktur Data. Dalam laporan ini dibahas mengenai implementasi struktur data queue menggunakan array dalam bahasa pemrograman Java dengan studi kasus antrian pelanggan pada loket pelayanan.

Penulis menyadari bahwa dalam penyusunan laporan ini tidak terlepas dari bantuan, bimbingan, dan dukungan dari berbagai pihak.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat bagi pembaca dan menambah wawasan dalam memahami struktur data, khususnya queue.

Padang, 13 April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I	1
PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	1
BAB II.....	2
PEMBAHASAN	2
2.1 Praktikum “IterasiQueue_2511533030”.....	2
2.2 Praktikum “QueueArray_2511533030”	2
2.3 Praktikum “QueueArrayDriver_2511533030”.....	2
2.4 Pratikum “QueueLinkedList_2511533030.....	3
2.5 Pratikum “Reverse_2511533030”	3
BAB III.....	3
KESIMPULAN.....	3
3.1 Kesimpulan.....	3
DAFTAR PUSTAKA.....	4

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi menuntut sistem yang mampu mengelola data secara terstruktur dan efisien. Salah satu permasalahan yang sering muncul adalah bagaimana mengatur urutan pelayanan pelanggan agar berjalan secara adil dan sistematis. Konsep antrian (queue) menjadi solusi yang tepat karena menerapkan prinsip FIFO, di mana pelanggan yang datang lebih dahulu akan dilayani terlebih dahulu. Dengan mengimplementasikan queue menggunakan array, mahasiswa dapat memahami bagaimana data disimpan, diakses, dan dikelola dalam memori secara langsung. Selain itu, pemahaman ini menjadi dasar penting dalam pembuatan berbagai aplikasi seperti sistem pelayanan, sistem antrean rumah sakit, dan sistem pemrosesan data lainnya.

1.2 Tujuan Pratikum

1. Memahami konsep dasar struktur data queue (antrian)
2. Mengimplementasikan queue menggunakan array dalam bahasa Java
3. Menerapkan prinsip FIFO (First In First Out) dalam program
4. Membuat operasi dasar queue seperti:
 - Enqueue (menambah data)
 - Dequeue (menghapus data)
 - Display (menampilkan data)
 - Pengecekan kondisi (kosong/penuh)

1.3 Manfaat Pratikum

1. Mahasiswa memahami cara kerja struktur data queue secara teori dan praktik
2. Meningkatkan kemampuan pemrograman menggunakan bahasa Java
3. Melatih logika dalam pengolahan data secara terstruktur
4. Membantu memahami penerapan queue dalam kehidupan nyata

BAB II PEMBAHASAN

2.1 Pratikum “IterasiQueue_2511533030”

Langkah Kerja :

1. Buat projek, package, dan class baru
2. Buat kode program berikut

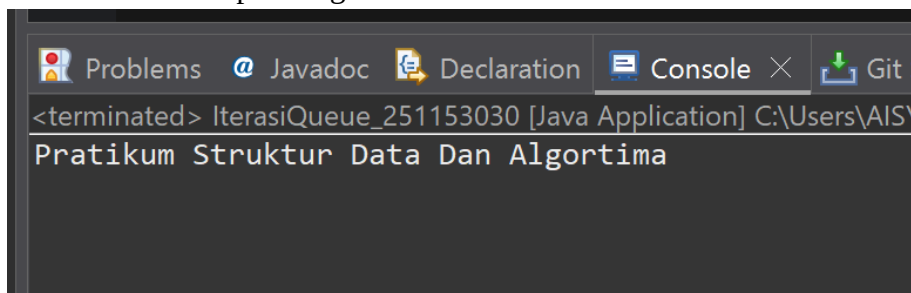
```
package pekan4_2511533030;
import java.util.Iterator;
import java.util.Queue;
import java.util.LinkedList;
public class IterasiQueue_2511533030 {
    public static void main (String args[])
    {
        Queue<String> q_3030 = new LinkedList<>();

        q_3030.add("Pratikum");
        q_3030.add("Struktur");
        q_3030.add("Data");
        q_3030.add("Dan");
        q_3030.add("Algoritma");
        Iterator<String> iterator = q_3030.iterator();
        while (iterator.hasNext()) {
            System.out.print(iterator.next() + " ");
        }

    }
}
```

Gambar 2.1 kode program “IterasiQueue_2511533030”

3. Output Program



Gambar 2.1 output program “IterasiQueue_2511533030”

Penjelasan kode :

2.1.1 Package dan Import

- Queue : Interface antrian (FIFO)
- LinkedList : Implementasi dari queue
- Iterator : Untuk mengakses elemen satu persatu

```
• package pekan4_2511533030;
• import java.util.Iterator;
• import java.util.Queue;
• import java.util.LinkedList;
```

2.1.2 Deklarasi Class

```
public class IterasiQueue_2511533030 {
```

2.1.3 Method main()

- Ini Adalah titik awal program dijalankan
`public static void main (String args[])`

2.1.4 Membuat Queue

- Membuat antrian bernama q_3030
- Tipe data : String
- Menggunakan LinkedList sebagai implementasi
`Queue<String> q_3030 = new LinkedList<>();`

2.1.5 Menambahkan Data (enqueue)

- Add() : Menambahkan data ke belakang antrian
- ```

• q_3030.add("Pratikum");
• q_3030.add("Struktur");
• q_3030.add("Data");
• q_3030.add("Dan");
• q_3030.add("Algortima");

```

#### 2.1.6 Membuat iterator

- Iterator digunakan untuk mengakses isi queue satu persatu
- Mulai dari elemen paling depan  
`Iterator<String> iterator = q_3030.iterator();`

#### 2.1.7 Perulangan (Iterasi)

- hasNext() : Cek apakah masih ada data
- next() : Ambil data berikutnya

```

• while (iterator.hasNext()) {
• System.out.print(iterator.next() + "
• ");
• }

```

## 2.2 Pratikum "QueueArray\_2511533030"

Langkah Kerja :

1. Buat class baru
2. Buat kode program berikut

```

package pekan4_2511533030;

public class QueueArray_2511533030 {
 int front_3030, rear_3030, size_3030;
 int capacity_3030;
 int array_3030[];

 public QueueArray_2511533030(int capacity) {
 this.capacity_3030 = capacity;
 front_3030 = this.size_3030 = 0;
 rear_3030 = capacity - 1;
 array_3030 = new int[this.capacity_3030];
 }

 boolean isFull_2511533030(QueueArray_2511533030 queue) {
 return (queue.size_3030 == queue.capacity_3030);
 }
}

```

```

boolean isEmpty(QueueArray_2511533030 queue) {
 return (queue.size_3030 == 0);
}

void enqueue(int item) {
 if (isFull_2511533030(this))
 return;
 this.rear_3030 = (this.rear_3030 + 1) % this.capacity_3030;
 this.array_3030[this.rear_3030] = item;
 this.size_3030 = this.size_3030 + 1;
 System.out.println(item + "enqueue to queue");
}

int dequeue_3030() {
 if (isEmpty(this))
 return Integer.MIN_VALUE;
 int item = this.array_3030[this.front_3030];
 this.front_3030 = (this.front_3030 + 1) % this.capacity_3030;
 this.size_3030 = this.size_3030 - 1;
 return item;
}

int front_3030() {
 if (isEmpty(this))
 return Integer.MIN_VALUE;
 return this.array_3030[this.front_3030];
}

int rear_3030() {
 if (isEmpty(this))
 return Integer.MIN_VALUE;
 return this.array_3030[this.rear_3030];
}

// Mencetak elemen antrian
void display_3030() {
 int i;
 if (front_3030 == rear_3030) {
 System.out.printf("\nAntrian Kosong\n");
 return;
 }

 // Kunjungi dari belakang dan cetak
 for (i = front_3030; i < rear_3030; i++) {
 System.out.printf(" %d <-- ", array_3030[i]);
 }
 return;
}
}

```

**Gambar 2.2 kode program “QueueArray\_2511533030”**

Penjelasan kode :

### 2.2.1 Deklarasi class

Mengimplementasikan struktur data queue(antrian) menggunakan array

```
public class QueueArray_2511533030 {
```

#### 2.2.2 Variabel

- **front\_3030** → indeks elemen paling depan (yang akan keluar duluan)
- **rear\_3030** → indeks elemen paling belakang (tempat masuk data baru)
- **size\_3030** → jumlah elemen dalam queue
- **capacity\_3030** → kapasitas maksimal queue
- **array\_3030[]** → tempat menyimpan data

```
• int front_3030, rear_3030, size_3030;
• int capacity_3030;
• int array_3030[];
```

#### 2.2.3 Constructor

- Mengatur kapasitas queue
- front = 0 → awal antrian
- rear = capacity - 1 → supaya enqueue pertama jadi index 0 (circular)
- Membuat array sesuai kapasitas

```
• public QueueArray_2511533030(int capacity) {
• this.capacity_3030 = capacity;
• front_3030 = this.size_3030 = 0;
• rear_3030 = capacity - 1;
• array_3030 = new int[this.capacity_3030];
• }
```

#### 2.2.4 Method isFull()

- Mengecek apakah antrian **penuh**
- Jika jumlah data = kapasitas → true

```
• boolean isFull_2511533030(QueueArray_2511533030 queue) {
• return (queue.size_3030 == queue.capacity_3030);
• }
```

#### 2.2.5 Method isEmpty()

- Mengecek apakah antrian kosong

```
• boolean isEmpty(QueueArray_2511533030 queue) {
• return (queue.size_3030 == 0);
• }
```

#### 2.2.6 Method enqueue()

- Menambahkan data ke belakang antrian
- Jika penuh → tidak ditambahkan
- rear = (rear + 1) % capacity → **circular queue**
- Data dimasukkan ke array
- Size bertambah



```

• void enqueue(int item) {
• if (isFull_2511533030(this))
• return;
• this.rear_3030 = (this.rear_3030 + 1) %
this.capacity_3030;
• this.array_3030[this.rear_3030] = item;
• this.size_3030 = this.size_3030 + 1;
• System.out.println(item + "enqueue to queue");
• }

```

#### 2.2.7 Method dequeue()

- Menghapus data dari depan
- Jika kosong → return nilai minimum
- Ambil data dari front
- Geser front ke depan
- Size berkurang

```

• int dequeue_3030() {
• if (isEmpty(this))
• return Integer.MIN_VALUE;
• int item = this.array_3030[this.front_3030];
• this.front_3030 = (this.front_3030 + 1) %
this.capacity_3030;
• this.size_3030 = this.size_3030 - 1;
• return item;
• }

```

#### 2.2.8 Method front()

- Mengambil elemen depan tanpa menghapus

```

• int front_3030() {
• if (isEmpty(this))
• return Integer.MIN_VALUE;
• return this.array_3030[this.front_3030];
• }

```

#### 2.2.9 Method rear()

- Mengambil elemen belakang

```

• int rear_3030() {
• if (isEmpty(this))
• return Integer.MIN_VALUE;
• return this.array_3030[this.rear_3030];
• }

```

#### 2.2.10 Method display()

- Menampilkan isi antrian

```

• // Mencetak elemen antrian
• void display_3030() {
• int i;
• if (front_3030 == rear_3030) {
• System.out.printf("\nAntrian Kosong\n");
• return;
• }
• }

```

```

• // Kunjungi dari belakang dan cetak
• for (i = front_3030; i < rear_3030; i++) {
• System.out.printf(" %d <-- ",
• array_3030[i]);
• }
• return;
• }
•

```

## 2.3 Pratikum “QueueArrayDriver\_2511533030”

Langkah kerja :

1. Buat Class baru
2. Buat kode berikut

```

package pekan4_2511533030;

public class QueueArrayDriver_2511533030 {
 public static void main(String[] args) {
 QueueArray_2511533030 queue = new QueueArray_2511533030(1000);
 queue.enqueue(10);
 queue.enqueue(20);
 queue.enqueue(30);
 queue.enqueue(40);
 System.out.println("Item di depan " + queue.front_3030());
 System.out.println("Item paling belakang " +
queue.rear_3030());
 System.out.println("tampilan queue");
 queue.display_3030();
 System.out.println();
 System.out.println(queue.dequeue_3030() + "dihapus dari
queue");
 System.out.println("Item di depan " + queue.front_3030());
 System.out.println("Item paling belakang " +
queue.rear_3030());
 System.out.println("tampilan queue setelah satu data
dihapus");
 queue.display_3030();
 }
}

```

Gambar 2.3 kode program “QueueArrayDriver\_2511533030”

3. Ditampilkan output sebagai berikut

```

<terminated> QueueArrayDriver_2511533030 [Java Applic
10enqueue to queue
20enqueue to queue
30enqueue to queue
40enqueue to queue
Item di depan 10
Item paling belakang 40
tampilan queue
 10 <-- 20 <-- 30 <--
10dihapus dari queue
Item di depan 20
Item paling belakang 40
tampilan queue setelah satu data dihapus
 20 <-- 30 <--

```

Gambar 2.3 output program “QueueArrayDriver\_2511533030”

Penjelasan Kode :

### 2.3.1 Deklarasi Class

```

public class QueueArrayDriver_2511533030 {

```

### 2.3.2 Method main()

- Titik awal program dijalankan
- Semua proses ada di sini

```
public static void main(String[] args) {
```

### 2.3.3 Membuat Objek Queue

- Membuat queue dengan kapasitas **1000 data**
- queue adalah objek dari class queue yang kamu buat

```
QueueArray_2511533030 queue = new
QueueArray_2511533030(1000);
```

### 2.3.4 Enqueue (Menambahkan Data)

- Menambahkan data ke antrian (FIFO)
- Urutan queue sekarang:

```
• queue.enqueue(10);
• queue.enqueue(20);
• queue.enqueue(30);
• queue.enqueue(40);
```

### 2.3.5 Menampilkan elemen depan dan belakang

- front\_3030() → mengambil data paling depan
- rear\_3030() → mengambil data paling belakang

```
• System.out.println("Item di depan " + queue.front_3030());
• System.out.println("Item paling belakang "
+ queue.rear_3030());
```

### 2.3.6 Menampilkan isi queue

```
System.out.println("tampilan queue");
queue.display_3030();
```

### 2.3.7 Dequeue (menghapus data)

- Menghapus data paling depan (FIFO)
- Yang dihapus: **10**

```
System.out.println(queue.dequeue_3030() + "dihapus dari
queue");
```

### 2.3.8 Menampilkan kode setelah dihapus

```
System.out.println("Item di depan " + queue.front_3030());
System.out.println("Item paling belakang " +
queue.rear_3030());
```

### 2.3.9 Menampilkan Queue setelah penghapusan

```
3. System.out.println("tampilan queue setelah satu data dihapus");
4. queue.display_3030();
```

## 2.4 Pratikum “QueueLinkedList\_2511533030”

Langkah kerja :

1. Buat Class baru

2. Buat kode berikut

```
1 package pekan4_2511533030;
2 import java.util.Queue;
3
4 public class QueueLinkedList_2511533030 {
5 public static void main(String[] args) {
6 Queue<Integer> q_3030 = new LinkedList<>();
7 // tambah elemen {0, 1, 2, 3, 4, 5} ke antrian
8 for (int i = 0; i < 6; i++)
9 q_3030.add(i);
10 // Menampilkan isi antrian.
11 System.out.println("Elemen Antrian " + q_3030);
12 // Untuk menghapus kepala antrian.
13 int hapus = q_3030.remove();
14 System.out.println("Hapus elemen = + hapus");
15 System.out.println(q_3030);
16 // Untuk melihat antrian terdepan
17 int depan = q_3030.peek();
18 System.out.println("Kepala Antrian = " + depan);
19
20 int banyak = q_3030.size();
21 System.out.println("Size Antrian = " + banyak);
22 }
23 }
24 }
25
```

Gambar 2.4 kode program “QueueLinkedList\_2511533030”

3. Ditampilkan output sebagai berikut

```
<terminated> QueueLinkedList_2511533030 [p
Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus elemen = + hapus
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size Antrian = 5
```

Gambar 2.4 output program “QueueLinkedList\_2511533030”

Penjelasan Kode :

2.4.1 Package & Import

- Queue → interface untuk struktur data antrian (FIFO)
- LinkedList → class yang digunakan sebagai implementasi Queue

```
• package pekan4_2511533030;
• import java.util.Queue;
• import java.util.LinkedList;
```

2.4.2 Deklarasi Class & main

- Class utama untuk menjalankan program
- main() adalah titik awal program

```
• public class QueueLinkedList_2511533030 {
• public static void main(String[] args) {
```

2.4.3 Membuat Queue

- Membuat queue dengan tipe data **Integer**

- Menggunakan LinkedList sebagai penyimpanan

```
Queue<Integer> q_3030 = new LinkedList<>();
```

#### 2.4.4 Menambahkan Elemen (enqueue)

- Menambahkan angka 0 sampai 5

```
• // tambah elemen {0, 1, 2, 3, 4, 5} ke antrian
• for (int i = 0; i < 6; i++)
• q_3030.add(i);
```

#### 2.4.5 Menampilkan Queue

```
// Menampilkan isi antrian.
System.out.println("Elemen Antrian " + q_3030);
```

#### 2.4.6 Menghapus Elemen (dequeue)

- Menghapus elemen paling depan (FIFO)
- Yang dihapus: 0

```
• // Untuk menghapus kepala antrian.
• int hapus = q_3030.remove();
```

#### 2.4.7 Menampilkan Queue setelah hapus

```
3 System.out.println("Hapus elemen = + hapus");
4 System.out.println(q_3030);
```

#### 2.4.8 Melihat elemen depan (peek)

- peek() → melihat elemen depan tanpa menghapus

```
• // Untuk melihat antrian terdepan
• int depan = q_3030.peek();
• System.out.println("Kepala Antrian = " + depan);
```

#### 2.4.9 Menghitung jumlah elemen

```
3 int banyak = q_3030.size();
4 System.out.println("Size Antrian = " + banyak);
```

## 2.5 Pratikum “ReverseData\_2511533030”

Langkah Kerja :

1. Buat Class baru
2. Buat kode program berikut

```
package pekan4_2511533030;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;
public class ReverseData_2511533030 {
 public static void main(String[] args) {
 Queue<Integer> q_3030 = new LinkedList<Integer>();
 q_3030.add(1);
 q_3030.add(2);
 q_3030.add(3); // [1, 2, 3]
 System.out.println("sebelum reverse" + q_3030);
 Stack<Integer> s = new Stack<Integer>();
 while (!q_3030.isEmpty()) { // Q -> S
 s.push(q_3030.remove());
 }
 }
}
```

```

 while (!s.isEmpty()) {
 q_3030.add(s.pop());
 }
 System.out.println("sesudah reverse= " + q_3030); // [3, 2, 1]
 }
}

```

**Gambar 2.5 kode program “ReverseData\_2511533030”**

3. Didapatkan output sebagai berikut

```

<terminated> QueueLinkedList_2511533030 [ja
Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus elemen = + hapus
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size Antrian = 5

```

**Gambar 2.5 output program “ReverseData\_2511533030”**

Langkah Kerja :

#### 2.5.1 Import Library

- Queue → struktur data antrian (FIFO)
- LinkedList → implementasi Queue
- Stack → struktur data tumpukan (LIFO)

```

import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

```

#### 2.5.2 Deklarasi Class & main

- Program utama dimulai dari main()

```

public class ReverseData_2511533030 {
 public static void main(String[] args) {

```

#### 2.5.3 Membuat Queue

- Membuat antrian bertipe integer

```
Queue<Integer> q_3030 = new LinkedList<Integer>();
```

#### 2.5.4 Menambahkan data

```

q_3030.add(1);
q_3030.add(2);
q_3030.add(3); // [1, 2, 3]

```

#### 2.5.5 Menampilkan sebelum reverse

```
System.out.println("sebelum reverse" + q_3030);
```

#### 2.5.6 Membuat stack

- Stack digunakan untuk **membalik urutan**
- Karena sifatnya **LIFO (Last In First Out)**

```
Stack<Integer> s = new Stack<Integer>();
```

#### 2.5.7 Proses reverse ( Queue ke Stack)

- Ambil dari queue (depan)
- Masukkan ke stack

```
while (!q_3030.isEmpty()) { // Q -> S
 s.push(q_3030.remove());
}
```

#### 2.5.8 Proses reverse (Stack ke Queue)

- Ambil dari stack (terakhir masuk)
- Masukkan kembali ke queue

```
• while (!s.isEmpty()) {
• q_3030.add(s.pop());
• }
```

#### 2.5.9 Menampilkan hasil

```
System.out.println("sesudah reverse= " + q_3030); // [3, 2, 1]
```

## BAB III KESIMPULAN

### 3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data **Queue (antrian)** merupakan salah satu struktur data yang bekerja berdasarkan prinsip **FIFO (First In First Out)**, yaitu data yang pertama masuk akan menjadi data pertama yang keluar.

Dalam praktikum ini, implementasi queue dilakukan dengan dua pendekatan, yaitu menggunakan **array (QueueArray)** dan menggunakan **LinkedList (Queue bawaan Java)**. Pada implementasi menggunakan array, mahasiswa dapat memahami secara lebih mendalam bagaimana proses pengelolaan data dilakukan secara manual, termasuk pengaturan indeks front, rear, kapasitas, serta penggunaan konsep **circular queue**. Sedangkan pada implementasi menggunakan LinkedList, proses pengelolaan queue menjadi lebih sederhana karena telah disediakan langsung oleh Java.

Selain itu, praktikum ini juga mencakup berbagai operasi dasar pada queue, seperti:

- **Enqueue** (menambahkan data ke antrian)
- **Dequeue** (menghapus data dari antrian)
- **Display** (menampilkan isi antrian)
- **Pengecekan kondisi** (kosong dan penuh)

Mahasiswa juga mempelajari penggunaan **Iterator** untuk menelusuri isi queue serta implementasi **reverse data**, yaitu membalik urutan antrian dengan bantuan struktur data **Stack (LIFO)**.

Secara keseluruhan, praktikum ini memberikan pemahaman yang baik mengenai konsep, implementasi, dan penerapan struktur data queue dalam pemrograman Java, serta meningkatkan kemampuan logika dan pemecahan masalah dalam pengolahan data secara terstruktur.

#### DAFTAR PUSTAKA

Herbert Schildt.2014. *Java: The Complete Reference*. New York: McGraw-Hill Education.



TUGAS PRATAIKUM

```

1 package pekan4_2511533030;
2 import java.util.Scanner;
3
4 public class AntrianLoket_2511533030 {
5 String queue[];
6 int front_3030, rear_3030, max;
7
8 // constructor
9 public AntrianLoket_2511533030(int max) {
10 this.max = max;
11 queue = new String[max];
12 front_3030 = 0;
13 rear_3030 = -1;
14 }
15
16 // cek kosong
17 boolean isEmpty() {
18 return rear_3030 < front_3030;
19 }
20
21 // cek penuh
22 boolean isFull() {
23 return rear_3030 == max - 1;
24 }
25
26 // enqueue
27 void enqueue(String data) {
28 if (isFull()) {
29 System.out.println("Antrian penuh!");
30 } else {
31 rear_3030++;
32 queue[rear_3030] = data;
33 System.out.println("Data berhasil ditambahkan ke antrian");

```

```

34 }
35 }
36
37 // dequeue
38 void dequeue() {
39 if (isEmpty()) {
40 System.out.println("Antrian kosong!");
41 } else {
42 System.out.println(queue[front_3030] + " telah dilayani");
43 front_3030++;
44 }
45 }
46
47 // display
48 void display() {
49 if (isEmpty()) {
50 System.out.println("\nAntrian kosong\n");
51 } else {
52 System.out.println("Isi antrian:");
53 for (int i = front_3030; i <= rear_3030; i++) {
54 System.out.println((i - front_3030 + 1) + ". " + queue[i]);
55 }
56 }
57 }
58
59 // reverse (pakai stack)
60 void reverse() {
61 if (isEmpty()) {
62 System.out.println("Antrian kosong!");
63 return;
64 }

```

```

66 Stack<String> s = new Stack<>();
67
68 // pindah queue ke stack
69 for (int i = front_3030; i <= rear_3030; i++) {
70 s.push(queue[i]);
71 }
72
73 // ambil lagi ke queue (kebalik)
74 int i = front_3030;
75 while (!s.isEmpty()) {
76 queue[i] = s.pop();
77 i++;
78 }
79
80 System.out.println("Antrian berhasil dibalik");
81 }
82
83 // ===== MAIN =====
84 public static void main(String[] args) {
85 Scanner input = new Scanner(System.in);
86 AntrianLoket_2511533030 q = new AntrianLoket_2511533030(10);
87
88 int pilih;
89
90 do {
91 System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
92 System.out.println("1. Tambah Antrian");
93 System.out.println("2. Hapus Antrian");
94 System.out.println("3. Tampilkan Antrian");
95 System.out.println("4. Reverse");
96 System.out.println("5. Keluar");
97 System.out.print("Pilih menu: ");

```

```

98 pilih = input.nextInt();
99 input.nextLine(); // buang enter
100
101 switch (pilih) {
102 case 1:
103 System.out.print("Masukkan nama pelanggan: ");
104 String nama = input.nextLine();
105 q.enqueue(nama);
106 break;
107
108 case 2:
109 q.dequeue();
110 break;
111
112 case 3:
113 q.display();
114 break;
115
116 case 4:
117 q.reverse();
118 break;
119
120 case 5:
121 System.out.println("Program selesai");
122 break;
123
124 default:
125 System.out.println("Menu tidak valid!");
126 }
127 } while (pilih != 5);
128 }
129

```

## Output

```
<terminated> AntrianLoket_2511533030 [Java Applic
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: budi
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: andi
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. budi
2. andi
```

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Antrian berhasil dibalik
```

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
andi telah dilayani
```

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. budi
```

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 5
Program selesai
```